Weather Data Analytics Platform

Sarthak Pani
Regd. No: 2241016150

Project ID: 18

B.Tech – Computer Science & Engineering
Siksha 'O' Anusandhan Deemed to be University

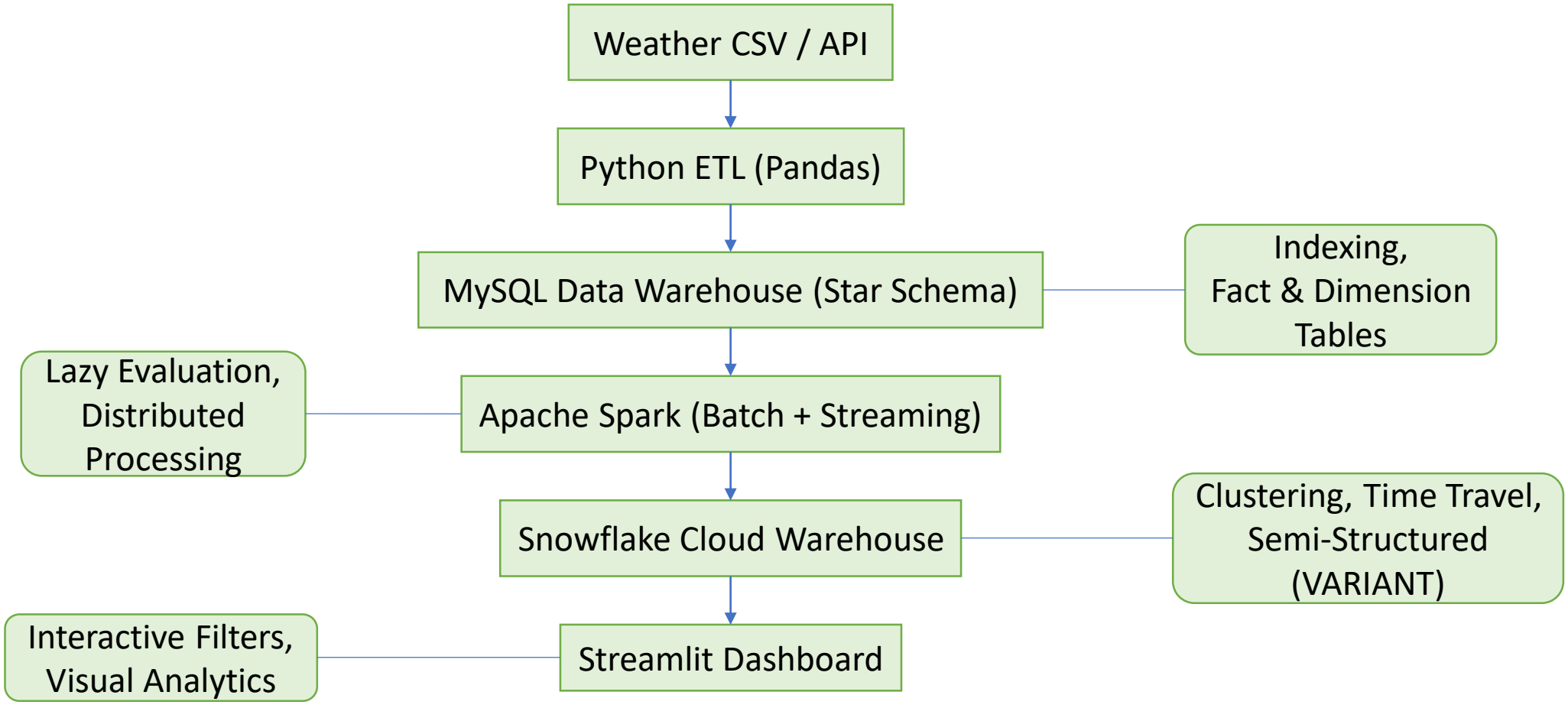
Problem Statement

- Weather data is generated continuously across multiple locations and time periods.
- Traditional OLTP systems are not optimized for large-scale analytical queries.
- An efficient Data Warehouse is required to support aggregation, trend analysis, and historical insights.
- Processing large datasets requires scalable tools like Spark and cloud-based platforms.
- A unified analytics platform is needed to transform raw data into meaningful insights.

Project Objectives

- Design a Star Schema Data Warehouse for structured analytical storage
- Implement Advanced SQL queries (CTE, Window Functions, Indexing, Partitioning)
- Develop a Python-based ETL pipeline for data extraction and transformation
- Process data using Apache Spark (Batch + Streaming)
- Store and optimize data in Snowflake (Clustering, Time Travel, Semi-Structured Data)
- Implement Security (RBAC, Data Masking, Governance)
- Perform Performance Tuning and Optimization
- Build an interactive Data Visualization Dashboard

System Architecture



Data Warehouse Design (Star Schema)

- Implemented a Star Schema for analytical querying
- Fact table stores measurable weather metrics (temperature, humidity, rainfall)
- Dimension tables store descriptive attributes (city, date)
- Reduced redundancy and improved query performance
- Optimized for aggregation and reporting

Fact Table:
- fact_weather

Dimension Tables:
- dim_city
- dim_date

Why Star Schema?
- Separates measurable data from descriptive data
- Reduces redundancy
- Improves aggregation performance
- Makes analytical queries faster and cleaner

Advanced SQL Implementation

- Used CTE (Common Table Expressions) for modular queries
- Applied Window Functions (RANK, ROW_NUMBER)
- Implemented Indexing for performance optimization
- Used GROUP BY and Aggregations for analytical reporting

Example:
RANK cities by average temperature using Window Function.

Python-Based ETL Pipeline

- Extracted weather data from CSV source
- Transformed data using Pandas (cleaning, formatting, date parsing)
- Created structured format aligned with Star Schema
- Loaded processed data into MySQL Data Warehouse
- Automated preprocessing logic for repeatability

Extract (CSV) → Transform (Pandas) → Load (MySQL)

Apache Spark Processing

- Used Spark for large-scale data processing
- Implemented Batch Processing for aggregations
- Demonstrated Streaming logic (conceptual implementation)
- Utilized Lazy Evaluation for optimized execution
- Applied transformations and actions for data computation

Key Concepts Used:

- Transformations (filter, groupBy, select)
- Actions (show, write, count)
- Narrow vs Wide transformations
- Distributed computation model

Snowflake Design & Optimization (Conceptual Implementation)

- Designed cloud data warehouse architecture using Snowflake (conceptual implementation)
- Implemented clustering for query performance optimization
- Used Time Travel feature for historical data recovery
- Handled semi-structured data using VARIANT data type
- Demonstrated Snowflake features for scalable analytical storage

Why Snowflake?

- Automatic scaling
- Cloud-native architecture
- No traditional indexing required
- Optimized for analytical workloads

Security & Governance

- Implemented Role-Based Access Control (RBAC)
- Defined separate roles for Admin and Analyst
- Applied data masking policies for sensitive fields
- Ensured controlled access to warehouse tables
- Maintained governance-ready architecture

Security Measures Applied:

- Role separation
- Permission management
- Controlled data visibility
- Secure data access practices

Performance Tuning & Optimization

- Applied indexing in MySQL for faster query execution
- Reduced redundancy using Star Schema design
- Leveraged Spark's lazy evaluation for optimized execution
- Designed clustering strategy in Snowflake for query optimization (conceptual)

Optimization Techniques Used:

- Index-based lookup (MySQL)
- Aggregation pushdown (Spark logical plan)
- Distributed processing (Spark)
- Micro-partition clustering (Snowflake)

Interactive Data Visualization Dashboard

- Built using Streamlit framework
- Displays weather trends across cities and time periods
- Implemented interactive filters (city selection, date range)
- Visualized KPIs using charts (bar chart, line graph)
- Enabled interactive analytical insights through dynamic filtering

Conclusion & Future Scope

Conclusion

- Successfully designed and implemented a Weather Data Analytics Platform
- Built an end-to-end pipeline from data ingestion to visualization
- Applied Data Warehouse concepts (Star Schema, Fact & Dimension modeling)
- Implemented advanced SQL, indexing, and optimization techniques
- Demonstrated Spark batch and streaming concepts
- Designed security and governance mechanisms (RBAC, data masking)
- Developed an interactive dashboard for analytical insights

Future Scope

- Deploy the solution on a cloud platform (AWS / Azure / GCP)
- Integrate real-time weather APIs instead of static CSV files
- Implement full Snowflake cloud deployment
- Add predictive analytics using Machine Learning
- Enable automated scheduling using Airflow or similar orchestration tools
- Enhance dashboard with advanced visualizations and alert systems